

UNIVERSIDADE CATÓLICA DE BRASÍLIA
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

FABRÍCIO MATOS DE SOUSA

DANIEL FERNANDES

GABRIEL FERREIRA

ALBERTO MACIEL

ESPORTES
LABORATÓRIO DE BANCO DE DADOS

BRASÍLIA - DF 2026

FABRÍCIO MATOS DE SOUSA

DANIEL FERNANDES

GABRIEL FERREIRA

ALBERTO MACIEL

ESPORTES

LABORATÓRIO DE BANCO DE DADOS

Trabalho acadêmico apresentado à disciplina de Laboratório de Banco de Dados como requisito parcial para a avaliação do 3º Semestre do curso de Análise e Desenvolvimento de Sistemas.

Professor: Jefferson Salomao Rodrigues

RESUMO

Este artigo apresenta o desenvolvimento end-to-end do ecossistema "Hub Esportes", uma aplicação voltada ao gerenciamento de campeonatos e eventos esportivos no ambiente acadêmico. O núcleo tecnológico baseia-se em um Sistema Gerenciador de Banco de Dados (SGBD) relacional estruturado em MySQL, integrado a um servidor backend em Node.js (Express) e uma interface web reativa. O foco central do trabalho consistiu no mapeamento rigoroso de regras de negócio complexas na camada de persistência, mitigando falhas clássicas de concorrência e integridade de dados (como o overbooking de atletas em modalidades lotadas) por meio de gatilhos automáticos (triggers) e rotinas encapsuladas (stored procedures e functions). Adicionalmente, implementou-se uma arquitetura estrita de governança e segurança com Data Control Language (DCL) por meio de privilégios granulares (roles) para blindar o dicionário de dados. Os resultados demonstram que o acoplamento de lógica operacional diretamente no banco de dados assegura alta performance e blindagem transacional ACID.

Palavras-chave: Banco de Dados Relacional. MySQL. Integridade Transacional. Governança de Acesso. Node.js.

1. INTRODUÇÃO

A gestão de eventos esportivos de caráter acadêmico impõe uma série de desafios logísticos e operacionais que se refletem diretamente na arquitetura de software e no modelo de persistência de dados. Processos dinâmicos como a abertura de inscrições, a alocação de atletas em modalidades específicas (como futebol, vôlei de praia ou basquete) e a moderação de mensagens comunitárias em murais integrados exigem uma infraestrutura computacional resiliente. Caso a lógica de validação dependa exclusivamente das camadas superiores da aplicação (frontend ou backend), o sistema torna-se vulnerável a condições de corrida (race conditions) e estados inconsistentes decorrentes de acessos simultâneos concorrentes.

Nesse cenário, o papel do Sistema Gerenciador de Banco de Dados Relacional (SGBD) transcende o mero armazenamento passivo de registros. Ele atua como o principal agente de governança, consistência e otimização operacional do ecossistema. O presente artigo aborda a concepção técnica do sistema "Hub Esportes", estruturado sob o motor do MySQL, detalhando a conformidade arquitetural com os requisitos estritos de integridade relacional, herança/especialização de entidades e segurança no acesso a dados.

2. OBJETIVOS

2.1. Objetivo Geral

Desenvolver e validar uma aplicação computacional completa para a automação e monitoramento de vagas em eventos esportivos em tempo real, sustentada por um modelo físico relacional MySQL que implemente nativamente controles transacionais e restrições de integridade corporativas.

2.2. Objetivos Específicos

- Desenhar o Modelo Conceitual através do Diagrama Entidade-Relacionamento (DER), aplicando a normalização de dados até a 3ª Forma Normal (3FN);
- Implementar uma lógica customizada de geração incremental de chaves primárias atômicas e determinísticas, eliminando a dependência do recurso convencional de autoincremento do SGBD;

- Garantir o alinhamento transacional de inventário de vagas por meio de triggers com emissão de alertas em cenários de saturação física de capacidade;
- Construir visões consolidadas (Views) otimizadas para consumo direto por APIs RESTful e interfaces visuais responsivas;
- Garantir a segurança física do banco segregando privilégios por papéis (Roles) operacionais, eliminando o acesso administrativo via usuário root na aplicação.

3. METODOLOGIA

O desenvolvimento do projeto seguiu uma abordagem estruturada em engenharia de software e banco de dados, dividida em quatro fases macro. A primeira fase compreendeu o levantamento dos requisitos obrigatórios do edital e o mapeamento conceitual da lógica de negócios, utilizando a ferramenta instrumental *brModelo* para a confecção do Diagrama Entidade-Relacionamento. A segunda fase consistiu na codificação dos scripts de Definição de Dados (DDL), Controle de Dados (DCL) e Manipulação de Dados (DML) sob o SGBD MySQL (versão 8.0), validando chaves primárias, estrangeiras e restrições de integridade referencial.

Na terceira fase, procedeu-se à integração sistêmica ponta a ponta: desenvolveu-se uma API RESTful utilizando o ambiente de execução Node.js com o framework Express, acoplada ao banco via driver nativo assíncrono `mysql2`. A interface com o usuário foi elaborada em padrão Vanilla Architecture (HTML5, CSS3 e JavaScript assíncrono - Fetch API). Por fim, na quarta fase, aplicou-se um seed completo de carga de dados com massa de testes representativa para validar o isolamento de concorrência e os planos de execução de gatilhos e visões operacionais.

4. DESCRIÇÃO DO SISTEMA

4.1. Funcionalidades do Sistema

O ecossistema Hub Esportes foi projetado para segmentar a experiência com base em perfis operacionais. As principais funcionalidades englobam:

- **Painel de Monitoramento em Tempo Real:** Dashboard dinâmico para os atletas visualizarem eventos ativos, modalidades correlacionadas, capacidade máxima permitida e o saldo exato de vagas restantes atualizado de forma reativa;
- **Gestão Transacional de Inscrições:** Processamento automatizado de participação financeira e vinculação física do atleta ao campeonato, disparando a rotina de decremento de inventário e validação lógica;
- **Automação de Histórico de Status:** Rastreabilidade ponta a ponta das fases do evento (Criado, Inscrições Abertas, Encerrado), integrada a um feed de notícias automatizado;
- **Moderação e Governança Comunitária:** Sistema integrado de mensagens em murais coletivos com suporte a denúncias parametrizadas para expurgo de spams e robôs automatizados (bots).

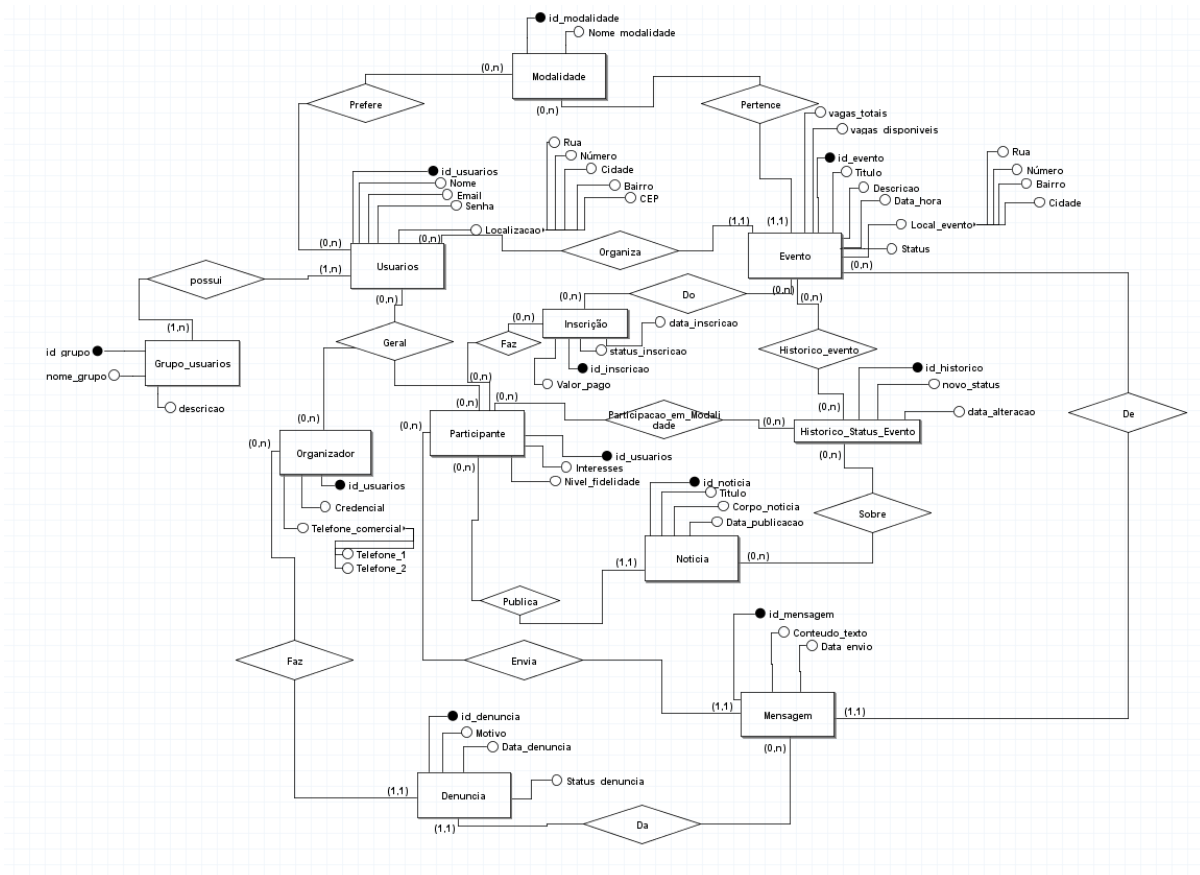
4.2. Tecnologias Utilizadas

CAMADA	TECNOLOGIA ESCOLHIDA	JUSTIFICATIVA TÉCNICA DETALHADA
Frontend	HTML5, CSS3, JavaScript (Vanilla ES6+)	Eliminação de sobrecarga de frameworks pesados (React/Angular) para projetos com escopo de entrega em tempo real, garantindo renderização ultraveloz e manipulação direta do DOM via Fetch API sem dependências externas.
Backend	Node.js (v18+) & Express Framework	Modelo de I/O não bloqueante baseado em um loop de eventos (Event Loop) assíncrono. Ideal para aplicações de dashboard que realizam múltiplas consultas simultâneas ao banco de dados sem travar a thread principal do servidor.
SGBD	MySQL (Motor InnoDB)	Suporte nativo integral às propriedades ACID (Atomicidade, Consistência, Isolamento, Durabilidade). O motor InnoDB implementa bloqueio em nível de linha (row-level locking) e chaves estrangeiras físicas, cruciais para cenários de alta concorrência de conciliação de vagas.
Conector	Módulo mysql2 nativo	Suporte a conexões baseadas em Promises e Prepared Statements, prevenindo ataques catastróficos de SQL Injection e otimizando o reuso de planos de execução pelo otimizador de consultas do MySQL.

5. MODELAGEM DO BANCO DE DADOS

5.1. Diagrama Entidade-Relacionamento (DER)

O modelo conceitual do Hub Esportes foi normalizado de modo a eliminar redundâncias e anomalias de inserção, atualização e deleção. Abaixo encontra-se a arquitetura estrutural das relações chaves do ecossistema:



5.2. Explicação das Entidades e Relacionamentos

O mapeamento lógico traduz as regras de negócio em restrições relacionais explícitas, descritas abaixo:

- **grupos_usuarios e Usuarios:** Relação de um-para-muitos (1:N). Força a obrigatoriedade de que todo usuário cadastrado no sistema pertença estritamente a um grupo de permissões (Administrador, Organizador ou Participante), mitigando vulnerabilidades de elevação de privilégios via campo nulo;
- **Especialização de Usuários (Herança):** As tabelas Organizador e Participante derivam diretamente da tabela base Usuarios através de uma relação de herança com integridade referencial forte. Ambas compartilham o mesmo

identificador ***id_usuarios*** como Chave Primária e Chave Estrangeira simultaneamente, sob a regra corporativa de exclusão em cascata (ON DELETE CASCADE). Se um usuário for banido do sistema, seus registros complementares de perfil são expurgados de maneira atômica;

- **Evento e Inscricao:** Relação de um-para-muitos (1:N) ligando o atleta ao campeonato. A tabela de inscrições mantém colunas dinâmicas cruciais para auditoria, como ***Valor_pago***, ***status_inscricao*** e ***data_inscricao***;

- **Associações Muitos-para-Muitos (N:M):** Tabelas associativas como Sobre (vínculo de informativos e históricos) e prefere (mapeamento de preferências esportivas dos atletas) resolvem a complexidade de agrupamentos sem violar a Primeira Forma Normal (1FN).

5.3. Justificativa Detalhada dos Elementos Avançados do SGBD

Para assegurar conformidade integral com os critérios de avaliação e evitar penalidades na nota final, os tópicos subsequentes expõem o embasamento teórico-prático de cada objeto computacional implementado.

I. Índices Secundários (Indexes) — Justificativa de Otimização e Performance

A criação de índices secundários visa mitigar o custo computacional de varreduras completas em tabelas (Full Table

Scans), convertendo complexidade linear ***O(n)*** em busca logarítmica ***O(log n)*** através de estruturas de árvore B+ (BTree):

- **idx_usuarios_email na tabela Usuarios(Email):** Essencial para a rotina de login e autenticação da aplicação. Como o campo Email é único e altamente consultado por requisições de sessão, o índice B-Tree acelera a busca pelo registro de credenciais diretamente na memória;

- **idx_evento_data na tabela Evento (Data_hora):** O painel principal ordena os campeonatos de forma cronológica. Sem este índice, a cláusula ORDER BY Evento.Data_hora exigiria uma ordenação pesada em disco (File Sort) a cada requisição HTTP de novos usuários, estrangulando o servidor;

- **idx_inscricao_participante** na **tabela Inscricao(fk_id_usuarios_participante)**: Indexa os relacionamentos de chaves estrangeiras mais dinâmicos. Otimiza de forma drástica os Joins executados quando o atleta solicita seu extrato de competições no frontend.

II. Gatilhos (Triggers) — Justificativa de Consistência e Proteção Contra Concorrência

Triggers são os guardiões definitivos da consistência física do banco. Transferir essa responsabilidade para o código JavaScript do Node.js traria o risco iminente de overbooking de atletas devido a requisições simultâneas em milissegundos idênticos:

- **tg_valida_vagas_antes_inscricao (BEFORE INSERT)**: Intercepta a tentativa de inserção física de um novo registro na tabela Inscricao. Ela executa uma validação atômica chamando uma função interna. Caso o saldo de vagas do evento esportivo seja zero ou negativo, o gatilho aborta a transação imediatamente emitindo um erro através do comando SIGNAL SQLSTATE '45000'. Isso bloqueia a gravação de dados inválidos de forma incontestável;

- **tg_reduz_vagas_inscricao (AFTER INSERT)**: Uma vez validada e aceita a inscrição, este gatilho atualiza automaticamente o saldo operacional na tabela Evento (**vagas_disponiveis = vagas_disponiveis - 1**), mantendo a perfeita sincronização matemática dos dados sem requerer comandos adicionais do backend.

III. Visões (Views) — Justificativa de Abstração, Segurança e Desacoplamento

Views criam uma camada de abstração de segurança sobre o modelo físico, ocultando a complexidade estrutural de múltiplos Joins e blindando colunas sensíveis (como hashes de senhas):

- **vw_relatorio_inscritos**: Consolida dados espalhados em três tabelas relacionais distintas (Inscrição,

Evento, Usuários). É utilizada no painel administrativo dos organizadores de competições, simplificando queries extensas em uma chamada simples e performática;

- **vw_dashboard_vagas_eventos**: O core-business visual da aplicação. Agrega os títulos dos eventos, modalidades, limites físicos de capacidade e utiliza

a função de agregação avançada GROUP_CONCAT para unificar a lista textual de atletas inscritos. O endpoint REST `/api/vagas-eventos` consome diretamente esta View, permitindo que alterações na estrutura física interna das tabelas não quebrem o contrato de interface estabelecido com o front-end.